System Design and Architecture

Student Names :

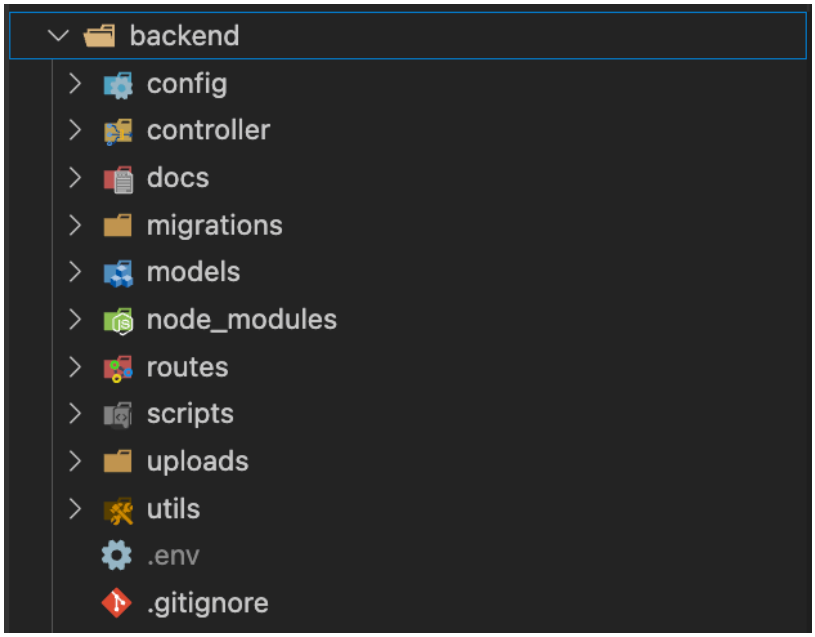
Harsh Doshi(92200133002) Krish Mamtora (92200133022) Rishit Rathod(92200133027)

Backend Folder Structure and Modularity

Our backend is organized into separate folders for controllers, models, routes, and configuration, following a modular structure:

1. **controller**/ — Contains logic for different modules like authentication, student management, faculty management, events, and analysis.
2. **models**/ — Defines the database structure for students, faculty, events, and batches.
3. **routes**/ — Handles API endpoints for different modules.
4. **config**/ — Contains configuration files such as database connection.

Backend folder structure:



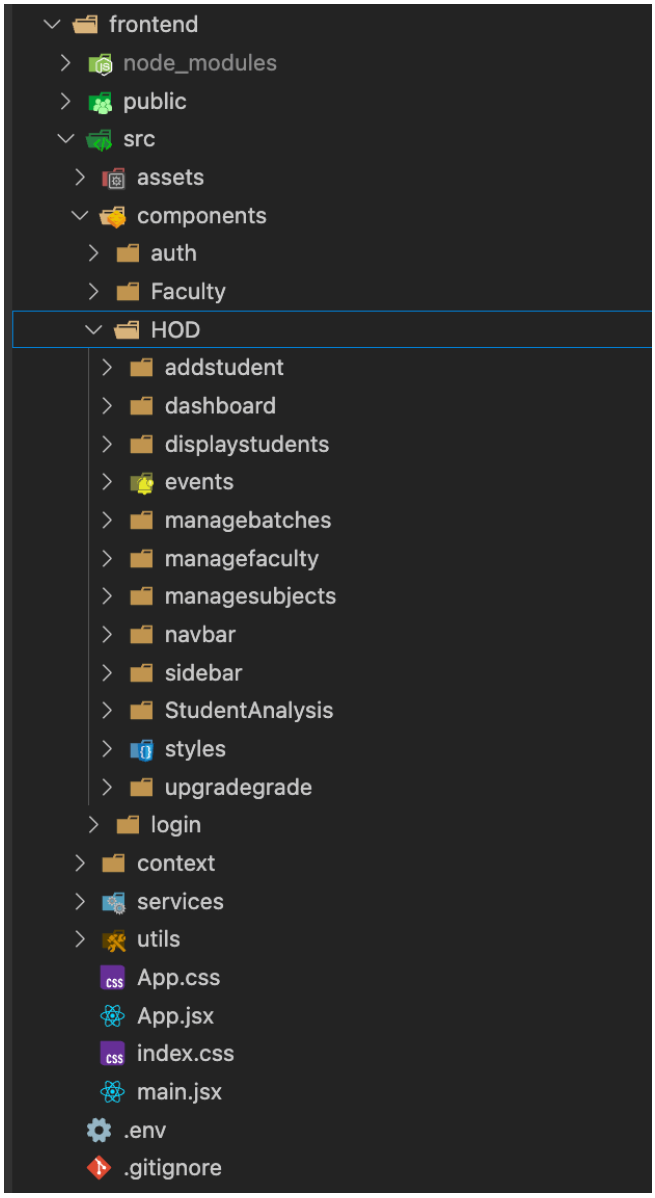
This structure makes the backend easy to maintain and extend. For example, if we want to add a new feature like attendance tracking, we can create a new controller, model, and route without disturbing existing modules.

Frontend Folder Structure and Modularity

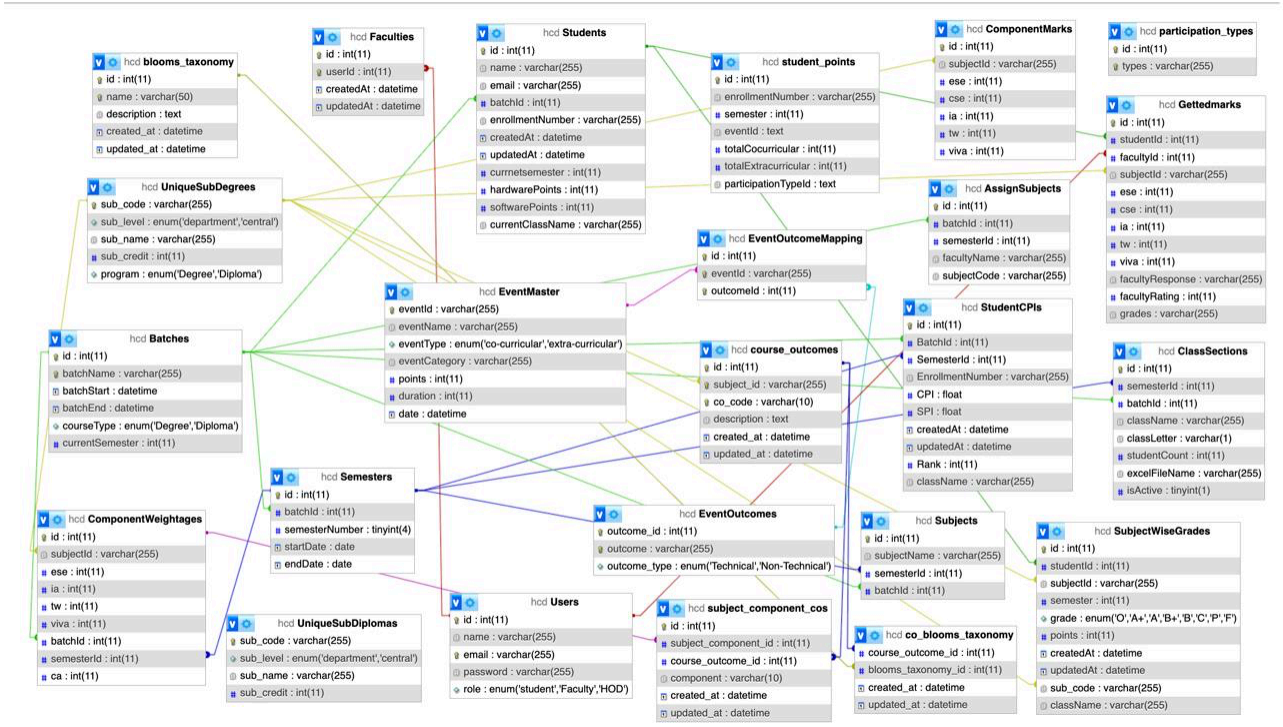
Our frontend is organized in a modular way, with all UI parts grouped inside the components/ folder based on their functionality and user roles.

- components/ — Contains all parts of the user interface, divided by features and user roles such as login, HOD dashboard, faculty dashboard, events, adding students, managing faculty and batches, and navigation components.

Frontend folder structure:



Key Features of the Database Schema Design



Organized Data Storage: The database has different tables like Students, Faculty, Events, Marks, Batches, and Subjects, keeping information neat and easy to manage.

Clear Connections: Tables are linked using keys, so related information can be easily found and used.

Easy to Expand: The design lets you add new tables or connections later without disturbing the existing system, making future upgrades simple.

Technology Stack

Our project is built using web and mobile tools that make it easy for HODs, teachers, students, and parents to use. We chose each tool because it is easy to work with, reliable, and matches the needs of the system.

Frontend Development

React.js (Web Portal for HOD & Faculty): We used React to build the web portal where HODs and teachers can see dashboards and student insights. It is fast and works well with charts for showing student data.

Flutter (Mobile App for Students & Parents): We chose Flutter to build one app that works on both Android and iOS. This saves time and gives students and parents the same smooth experience on any device.

Backend Development

Node.js & Express.js: The backend is built with Node and Express. It manages the data flow between the app, the website, and the database. We used it because it is quick, reliable, and can handle many users at once.

Database Management

MySQL: Our database is in a structured form, and we chose MySQL because we had worked with it before and found it easy to use.

Programming Languages

JavaScript: We used JavaScript for both the frontend and backend to keep the whole system in a single technology, making development easier and more consistent.

Dart: Flutter uses Dart, so we used it for building mobile apps with good performance and responsive design.

Visualization & Analytics

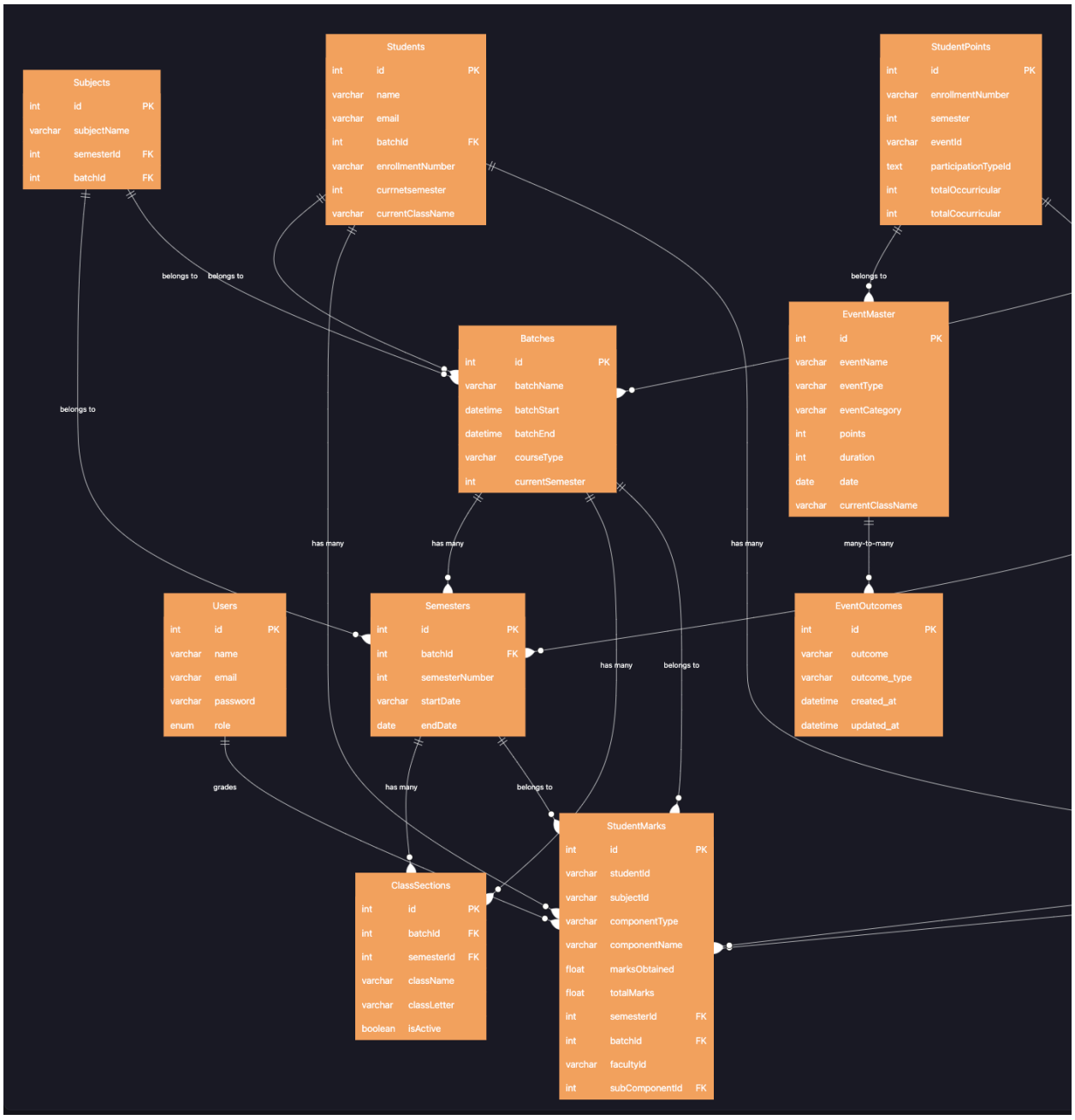
Chart.js & Recharts: These tools are used to make dashboards that show performance data in a clear and interactive way.

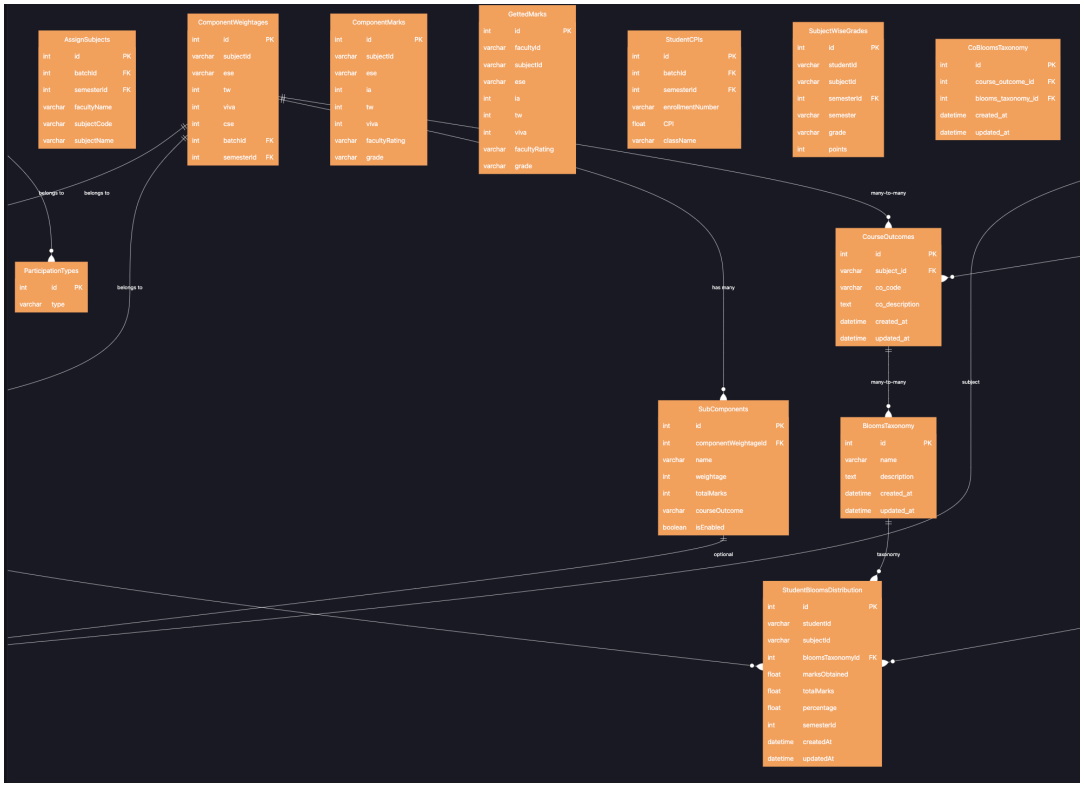
PDF Export: The system lets HODs easily download performance reports in PDF format.

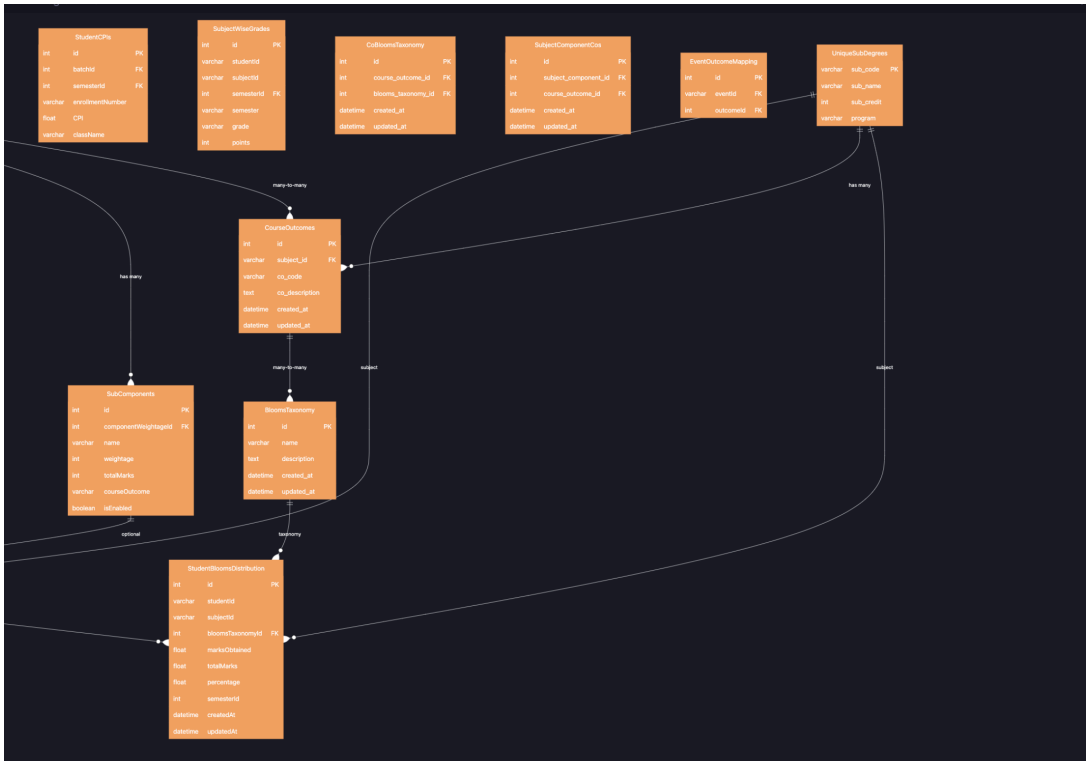
Version Control & Reporting

GitHub: We used GitHub to manage code, work as a team, and collaborate.
Email Reports: The system can send performance reports to students and parents through email, making communication faster and easier.

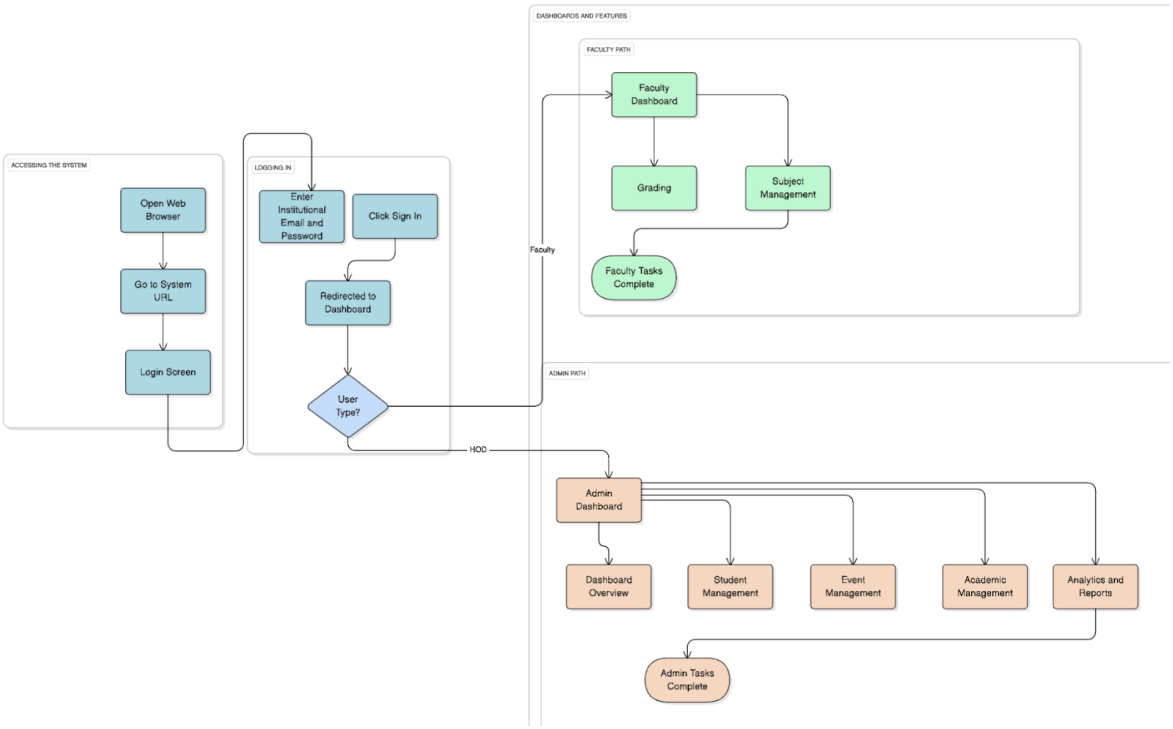
ER diagram


Subjects
int id PK
varchar subjectName
int semesterId FK
int batchId FK
Students
int id PK
varchar name
varchar email
int batchId FK
varchar enrollmentNumber
int currnetsemester
varchar currentClassName
StudentPoints
int id PK
varchar enrollmentNumber
int semester
varchar eventId
text participationTypeId
int totalOccurricular
int totalCocurricular
Batches
int id PK
varchar batchName
datetime batchStart
datetime batchEnd
varchar courseType
int currentSemester
EventMaster
int id PK
varchar eventName
varchar eventType
varchar eventCategory
int points
int duration
date date
varchar currentClassName
Users
int id PK
varchar name
varchar email
varchar password
enum role
Semesters
int id PK
int batchId FK
int semesterNumber
varchar startDate
date endDate
EventOutcomes
int id PK
varchar outcome
varchar outcome_type
datetime created_at
datetime updated_at
ClassSections
int id PK
int batchId FK
int semesterId FK
varchar className
varchar classLetter
boolean isActive
StudentMarks
int id PK
varchar studentId
varchar subjectId
varchar componentType
varchar componentName
float marksObtained
float totalMarks
int semesterId FK
int batchId FK
varchar facultyId
int subComponentId FK
belongs to
belongs to
belongs to
belongs to
has many
has many
has many
many-to-many
has many
belongs to
grades
has many
belongs to


AssignSubjects
ComponentWeightages
ComponentMarks
GetSetMarks
StudentCPIs
SubjectWiseGrades
CoBloomsTaxonomy
ParticipationTypes
CourseOutcomes
SubComponents
BloomsTaxonomy
StudentBloomsDistribution
belongs to
belongs to
belongs to
many-to-many
has many
many-to-many
subject
optional
taxonomy


StudentCPIs
SubjectWiseGrades
CoBloomsTaxonomy
SubjectComponentCos
EventOutcomeMapping
UniqueSubDegrees
CourseOutcomes
SubComponents
BloomsTaxonomy
StudentBloomsDistribution
many-to-many
has many
has many
many-to-many
subject
subject
optional
taxonomy

Flow chart


ACCESSING THE SYSTEM
Open Web Browser
Go to System URL
Login Screen
LOGGING IN
Enter Institutional Email and Password
Click Sign In
Redirected to Dashboard
User Type?
Faculty
HOD
DASHBOARDS AND FEATURES
FACULTY PATH
Faculty Dashboard
Grading
Subject Management
Faculty Tasks Complete
ADMIN PATH
Admin Dashboard
Dashboard Overview
Student Management
Event Management
Academic Management
Analytics and Reports
Admin Tasks Complete